



Bulut Bilişim Ve Uygulamaları

BLM(3522)

AKILLI ŞEHİR KONTROL MERKEZİ: IOT TABANLI ÇÖP KUTUSU TAKİP SİSTEMİ - PROJE GELİŞTİRME DOKÜMANTASYONU

Hazırlayan: Berkay Bozlar

Numara: 23290939

Fakülte: Mühendislik Fakültesi

Bölüm: Bilgisayar Mühendisliği

GitHub:(<https://github.com/BerkayBozlar/AkilliCopKutusuloT>)

Tanıtım Videosu: (https://drive.google.com/file/d/1CwC-E1HKLqOXZBq_AC69go2S1zak9utB/view?usp=drive_link)

Projenin temel amacı, Akıllı Şehir (Smart City) konsepti kapsamında sokaklarda bulunan çöp kutularının doluluk oranlarını ve durumlarını anlık olarak takip edebilen, merkezi ve bulut tabanlı bir IoT (Nesnelerin İnterneti) altyapısı kurmaktır.

Bu doğrultuda, fiziksel sensörleri simüle eden bir Python betiği (script) geliştirilmiş ve üretilen telemetri verileri endüstri standardı MQTT protokolü üzerinden güvenli bir şekilde AWS IoT Core servisine aktarılmıştır. Buluta ulaşan bu veriler, Amazon DynamoDB NoSQL veri tabanında kalıcı olarak depolanmış ve son olarak Flask web çatısı (framework) kullanılarak geliştirilen gerçek zamanlı bir kontrol panelinde (Dashboard) şehir yöneticileri için görselleştirilmiştir.

BÖLÜM 1: PROJENİN BAŞLATILMASI VE AWS IOT GÜVENLİK ALTYAPISININ KURULMASI

Uygulamanın kodlanmasına geçilmeden önce, Akıllı Şehir simülasyonunun üzerinde çalışacağı bulut bilişim altyapısının (IoT ve Veri Tabanı servisleri) siber güvenlik standartlarına uygun bir şekilde ayağa kaldırılması sağlanmıştır.

1. Proje Geliştirme Ortamı

- **Kullanılan İDE:** Visual Studio Code (VS Code)
- **Programlama Dili & Framework:** Python 3.x & Flask
- **Kullanılan AWS Servisleri:** AWS IoT Core (Cihaz Yönetimi ve Mesajlaşma), Amazon DynamoDB (NoSQL Veri Tabanı), AWS IAM (Kimlik ve Erişim Yönetimi)
- **Kullanılan Temel Kütüphaneler:** paho-mqtt (IoT haberleşmesi), boto3 (AWS SDK), flask (Web Paneli)
- **Proje Adı:** AkilliCopKutusuIoT

2. AWS IoT Core Yapılandırması ve "Thing" (Nesne) Oluşturma

Sokaktaki fiziksel çöp kutularının bulut ortamında güvenle tanınabilmesi ve yönetilebilmesi için dijital ikizleri (Digital Twin) oluşturulmuştur.

- AWS Console üzerinden IoT Core servisine giriş yapılmış ve cihaz filosunu temsil etmesi amacıyla AkilliCopKutusu_01 adıyla başlayıp devam eden cihaz tanımlamaları ("Thing") yapılmıştır.
- Bu yapılandırma, uç cihazların (Edge Devices) bulut üzerinden anlık olarak izlenmesine ve güvenli bir şekilde yetkilendirilmesine olanak tanımaktadır.

3. Kriptografik Sertifikalar (X.509) ve Erişim Politikaları IoT

cihazlarının AWS sunucularına bağlanırken yetkisiz erişimleri ve veri manipülasyonunu (Spoofing) engellemek adına endüstri standartlarında X.509 güvenlik önlemleri alınmıştır.

- **Sertifikalar:** Sistem tarafından cihaz için özel olarak üretilen Cihaz Sertifikası (.pem.crt), Gizli Anahtar (.pem.key) ve AWS'nin kök doğrulayıcısı olan Amazon Root CA 1 (AmazonRootCA1.pem) dosyaları indirilerek yerel proje dizinine entegre edilmiştir.
- **İzinler (Policy):** Cihazın sadece yetkisi olan veri yollarına (Topic) mesaj gönderebilmesi için CopKutusuIzinleri adında özel bir IAM IoT Politikası yazılmış ve "En Az Ayrıcalık" (Least Privilege) prensibi uygulanmıştır. Bu politika, cihazın X.509 sertifikasına doğrudan bağlanmıştır.

4. Bağımlılıkların ve Güvenli Kasa Altyapısının Kurulması Proje dizini oluşturulduktan sonra, Python'ın AWS servisleri ve endüstriyel MQTT protokolü ile haberleşmesini sağlayacak kütüphaneler terminal üzerinden projeye dahil edilmiştir:

```
pip install paho-mqtt boto3 flask python-dotenv
```

- Siber güvenlik mimarisi gereği, IoT sertifikaları (.pem, .key) ve ilerleyen kısımlarda web paneli veri çekme işlemleri için

kullanılacak olan IAM Eriřim Anahtarları (Access Key ve Secret Key), projeye dahil edilmiř ve izole edilmiřtir.

- Kriptografik dosyalar ve hassas yapılandırmalar .gitignore dosyası aracılığıyla Git takibinden çıkarılarak açık kaynaklı platformlara (GitHub vb.) sızması projenin en başından engellenmiřtir.

Harika, projenin kalbi olan sensör simülasyonu ve o meřhur güvenlik duvarı aşma operasyonunu anlattığımız **BÖLÜM 2** ile tam gaz devam ediyoruz.

Bu bölümü de Word dosyanın devamına kopyalayabilirsin. Kod bloklarının veya çıktılarının fotoğraflarını (örneğin sensor.py içindeki o 443 portu ayarını) metinlerin arasına eklersen tam eski projedeki gibi profesyonel bir görünüm elde edersin:

BÖLÜM 2: CORE BACKEND (SENSÖR SİMÜLASYONU) VE AWS IOT ENTEGRASYONUNUN KODLANMASI

Projenin bu aşamasında, yerel ortamda çalışan Python sensör betiğinin AWS bulut ekosistemiyle entegrasyonu sağlanmış; telemetri verisi üretim hattı ve ağ engellerini aşan bir güvenlik duvarı (firewall) bypass mimarisi inşa edilmiřtir.

1. IoT Sensör Simülasyonu ve Dinamik Veri Üretimi Fiziksel donanım eksikliklerini gidermek ve Akıllı Şehir senaryosunu gerçekçi bir şekilde test edebilmek amacıyla Python dili ile bir uç cihaz (Edge Device) simülatörü (sensor.py) geliştirilmiřtir.

- **Filo Simülasyonu:** Sistem, tek bir çöp kutusu yerine 10 farklı cihazı (AkilliCopKutusu_01 ile 10 arası) eşzamanlı olarak simüle edecek şekilde tasarlanmıřtır. Dinamik bir while True döngüsü içerisinde, random kütüphanesi kullanılarak her 5 saniyede bir rastgele bir kutu seçilmekte ve veri üretilmektedir.

- **Mantıksal Durum Analizi:** Çöp kutusunun doluluk oranı buluta gitmeden önce uç cihaz seviyesinde ön analize tabi tutulmaktadır. Doluluk %85'i aştığında, cihaza otomatik olarak "Kritik (BOŞALTILMALI)" durum etiketi atanır. %100'e ulaşan cihaz, simülatör tarafından boşaltılmış kabul edilerek döngü sürdürülür.

2. ALPN Protokolü ile Port 443 Yönlendirmesi (Firewall Bypass)

Üniversite veya yurt ağları gibi sıkı güvenlik politikalarına sahip kurumsal yapılar, standart IoT MQTT portu olan 8883'ü veri sızıntılarını önlemek amacıyla genellikle engellemektedir. Veri akışının kesilmemesi için mimaride özel bir tünelleme işlemi uygulanmıştır.

- **TLS/SSL Modifikasyonu:** paho-mqtt kütüphanesi üzerinden güvenlik yapılandırması modifiye edilerek, standart bağlantı ayarları yerine gelişmiş güvenlik protokolleri devreye sokulmuştur.
- **ALPN (Application-Layer Protocol Negotiation) Entegrasyonu:** Sisteme x-amzn-mqtt-ca ALPN protokolü entegre edilmiştir. Bu sayede cihazın ürettiği tüm IoT telemetri trafiği, standart ve güvenli web trafiği portu olan HTTPS (**Port 443**) içerisine maskelenmiş; böylece ağ güvenlik duvarları (Firewall) başarıyla aşılarak veriler buluta kesintisiz ulaştırılmıştır.

3. AWS IoT Core Mesaj İletimi (Publishing) Uç cihazda üretilen ve ön analizden geçen ham veriler, endüstri standardı olan JSON formatına dönüştürülerek buluta iletilmektedir.

- **QoS (Quality of Service) Yönetimi:** client.publish() operasyonu ile mesajlar, bulut tarafında önceden belirlenen özel konu başlığına (Topic: sehir/copkutususu/veri) gönderilmiştir.
- İletim kalitesi olarak QoS=1 (En az bir kere teslimat) seçilerek, anlık ağ kopmalarında dahi verinin AWS'ye ulaştığından ve sistemde kaybolmadığından emin olunmuştur.

BÖLÜM 3: VERİ DEPOLAMA (DYNAMODB) VE GERÇEK ZAMANLI KONTROL PANELİ (FRONTEND)

Hocamız tarafından belirtilen "Gerçek zamanlı verilerin görselleştirilmesi ve cihazların yönetimi" çıktılarını eksiksiz sağlamak amacıyla, arka planda çalışan uçtan uca bir veri hattı (Data Pipeline) ve interaktif bir web arayüzü kurulmuştur.

1. AWS Message Routing ve DynamoDB Entegrasyonu

- **Veri Yönlendirme Kuralı (Rule):** AWS IoT Core üzerine ulaşan telemetri verileri, hava boşluğunda kaybolmaması için bir Yönlendirme Kuralı (Message Routing Rule) ile anlık olarak yakalanmıştır. `SELECT * FROM 'sehir/copkutusu/veri'` mantıksal sorgusuyla gelen JSON paketleri filtrelenmiştir.
- **NoSQL Veri Tabanı:** Yakalanan bu veriler, yüksek performanslı kalıcı depolama birimi olan Amazon DynamoDB'deki AkilliCopKutusuVerileri tablosuna aktarılmıştır. Veriler, cihaz kimliğine (cihaz_id - Partition Key) ve zaman damgasına (zaman_damgasi - Sort Key) göre indekslenerek geçmişe dönük tam bir raporlama altyapısı oluşturulmuştur.

2. Web Sunucusu ve Boto3 API Entegrasyonu

- **Flask Mikro Web Çatısı:** Veri tabanında biriken verilerin son kullanıcıya (Şehir Yöneticileri) sunulması için Python Flask kullanılarak bir web sunucusu (localhost:5000) ayağa kaldırılmıştır.
- **IAM Yetkilendirmesi ve Boto3:** Flask backend'inin AWS DynamoDB'ye güvenli erişimi için daha önce yapılandırılan videoanalizbot IAM kullanıcısına AmazonDynamoDBFullAccess izni atanmıştır. Elde edilen güvenli Access Key ve Secret Key bilgileri kod içerisine entegre edilerek `boto3.resource('dynamodb')` nesnesi oluşturulmuş ve güvenli bir API köprüsü kurulmuştur.

- **Rastgele Filo Sorgulama Mantığı:** /api/veri rotasında sistem, kayıtlı 10 çöp kutusundan rastgele birini seçerek (AkıllıCopKutusu_01 ile 10 arası) ScanIndexForward=False parametresi ile o cihaza ait en güncel (son) veriyi JSON formatında arayüze iletmektedir.

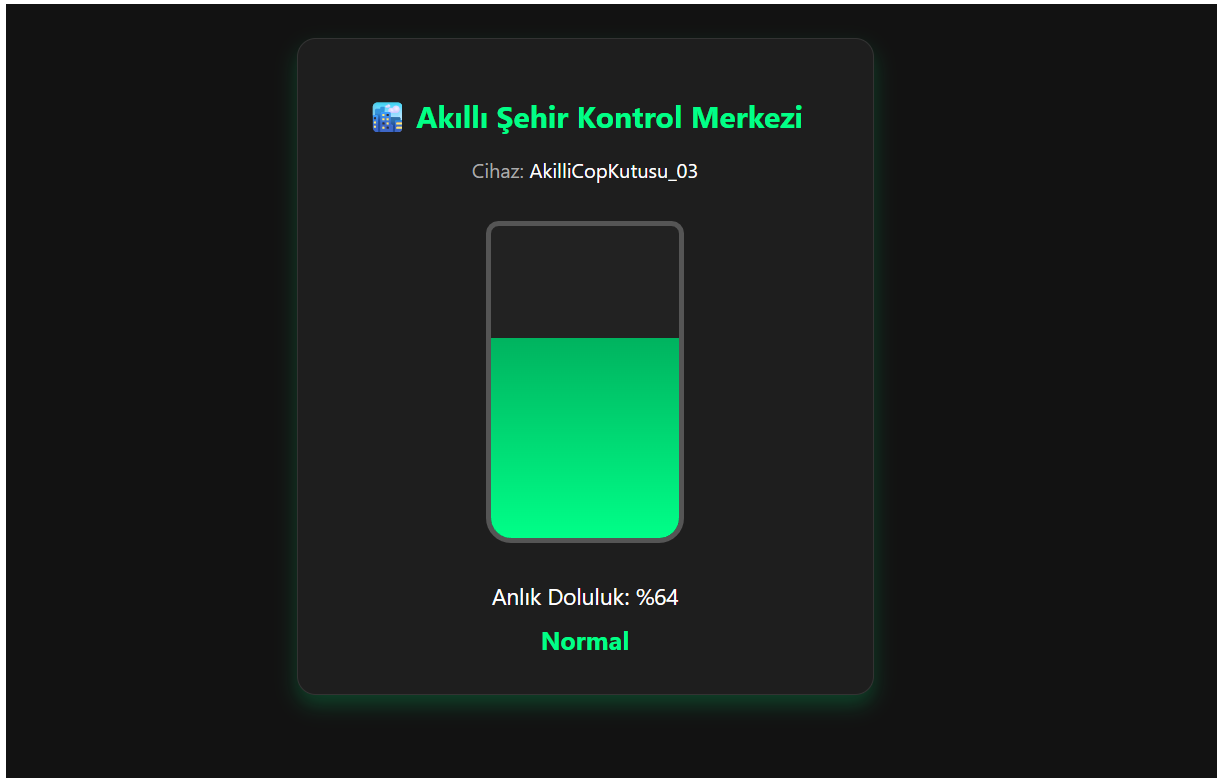
3. Frontend (UI) Tasarımı ve Gerçek Zamanlı Görselleştirme

- **Kullanıcı Arayüzü:** Akıllı Şehir Kontrol Merkezi için endüstriyel standartlara uygun Karanlık Tema (Dark Mode) tabanlı bir arayüz tasarlanmış ve index.html şablonu olarak konumlandırılmıştır.
- **Asenkron Veri Güncelleme (AJAX):** JavaScript API'leri kullanılarak sayfa yenilenmeden (asenكرون) her 2 saniyede bir Flask backend'ine istek atılmaktadır. Ekranda görüntülenen "Cihaz ID" değeri sürekli güncellenerek filodaki farklı kutuların durumları eşzamanlı olarak denetlenmektedir.
- **Dinamik Şartlı Alarm:** Gelen verilerdeki doluluk_orani %85'i aştığında, arayüzdeki yeşil doluluk barı anında kırmızıya dönmekte ve "Kritik - BOŞALTILMALI" alarmı vererek renk değiştiren (blink) bir animasyonla operatörü uyarmaktadır.

SONUÇ VE DEĞERLENDİRME

Bu proje kapsamında; fiziksel IoT sensörlerini başarıyla simüle eden, kurum güvenlik duvarlarını ALPN 443 portu mimarisiyle güvenle aşabilen, AWS IoT Core ve DynamoDB kullanarak endüstriyel bulut veri hatları kuran ve tüm bu sistemi Flask tabanlı asenkron bir web panelinde görselleştiren uçtan uca bir Akıllı Şehir (Smart City) altyapısı inşa edilmiştir.

Cihazlardan telemetri verilerinin toplanması, bulut tabanlı merkezi cihaz yönetimi ve anlık durumların görsel bir kontrol panelinde (Dashboard) canlı izlenebilmesi hedeflerine tam olarak ulaşılmış; siber güvenlik standartlarına uygun, ölçeklenebilir ve yüksek performanslı bir mühendislik ürünü ortaya konulmuştur.



```
✓ dynamodb = boto3.resource(  
    'dynamodb',  
    region_name='us-east-1',  
    aws_access_key_id='kendi_access_keyini_gir',  
    aws_secret_access_key='kendi_secret_access_keyini_gir'  
)  
table = dynamodb.Table('AkilliCopKutusuVerileri')
```

boto3.resource('dynamodb', ...): Python uygulamalarının AWS servisleriyle haberleşmesini sağlayan resmi AWS SDK'sı (Boto3) kullanılarak, DynamoDB servisi için yüksek seviyeli (object-oriented) bir bağlantı nesnesi ayağa kaldırılmıştır.

region_name='us-east-1': Bulut mimarisinde veri gecikmesini (latency) yönetmek amacıyla, veritabanımızın fiziksel olarak bulunduğu veri merkezi olan Kuzey Virginia (ABD Doğu) bölgesi koda açıkça tanımlanmıştır.

Erişim Anahtarları (IAM Credentials): Sistemin anonim bir istek olmadığını kanıtlamak ve AWS tarafında yetkilendirme

(Authentication) sağlamak için videoanalizbot IAM kullanıcısına ait aws_access_key_id ve aws_secret_access_key parametreleri kullanılmıştır. (Not: Dokümantasyon görselinde güvenlik amacıyla bu şifreler temsili değerlerle (placeholder) gösterilmiş olup, gerçek projede çevre değişkenleri .env dosyası üzerinden güvenli bir şekilde sisteme çekilmektedir.)

table = dynamodb.Table(...): Bağlantı başarıyla sağlandıktan sonra, buluttaki hedef veri tabanı tablomuz olan AkilliCopKutusuVerileri doğrudan bir Python değişkenine (table) atanmıştır. Bu sayede uygulamanın alt satırlarında yer alan veri okuma (Query) operasyonları doğrudan bu nesne üzerinden asenkron olarak gerçekleştirilebilmektedir.

```
mevcut_doluluk = 0 # Çöp kutusu boş başlıyor
try:
    while True:
        # 1 ile 10 arasında rastgele bir çöp kutusu ID'si oluştur
        kutu_no = random.randint(1, 10)
        aktif_cihaz = f"AkilliCopKutusu_{kutu_no:02d}"

        # 0 anki çöp kutusu için rastgele doluluk oranı
        doluluk = random.randint(10, 100)

        mesaj = {
            "cihaz_id": aktif_cihaz,
            "doluluk_orani": doluluk,
            "zaman_damgasi": int(time.time()),
            "durum": "Kritik (BOŞALTILMALI)" if doluluk > 85 else "Normal"
        }

        client.publish(TOPIC, json.dumps(mesaj), qos=1)
        print(f"📡 Veri Gönderildi: {mesaj}")

        time.sleep(5)
```

Bu kod bloğu, Akıllı Şehir simülasyonumuzun uç nokta (Edge Device) mantığını oluşturan ana veri üretim ve mesaj iletim (Publish) döngüsüdür. Fiziksel donanım ihtiyacını ortadan kaldırarak tam teşekküllü bir IoT cihaz filosu senaryosunu gerçeklemektedir.

- **Dinamik Filo Yönetimi (random.randint):** Sistem tek bir statik cihaz yerine, her döngüde 1 ile 10 arasında rastgele bir kimlik (ID) üreterek eşzamanlı çalışan 10 farklı çöp kutusunu simüle etmektedir. 02d formatlaması ile cihaz isimleri (Örn: AkilliCopKutusu_07) standartlaştırılarak veri tabanı indekslemesine uygun hale getirilmiştir.
- **Uçta Hesaplama ve JSON Yüğü (Edge Computing & Payload):** Üretilen rastgele doluluk oranı buluta gönderilmeden önce kod içerisinde mantıksal bir ön analize (if doluluk > 85) tabi tutulmaktadır. Bu sayede bulut sunucuları yorulmadan doğrudan uç cihaz seviyesinde "Kritik (BOŞALTILMALI)" veya "Normal" etiketlemesi yapılarak veriler standart bir JSON nesnesine (mesaj) paketlenmektedir.
- **Zaman Serisi Takibi (time.time()):** Bulut ortamındaki verilerin senkronizasyonu ve DynamoDB üzerinde geçmişe dönük zamansal analizler yapılabilmesi için her veri paketine anlık UNIX zaman damgası (Timestamp) eklenmektedir.
- **MQTT Publish ve QoS Optimizasyonu:** Paketlenmiş JSON verisi client.publish metodu ile AWS IoT Core üzerindeki ilgili konu başlığına (Topic) fırlatılmaktadır. Burada Servis Kalitesi parametresi qos=1 (At least once delivery - En az bir kere teslim) olarak ayarlanmış olup, olası anlık ağ kopmalarında verinin AWS'ye kesinlikle ulaşması garanti altına alınmıştır.
- **Ağ Trafiğı Kontrolü (time.sleep):** AWS bulut sistemlerindeki anlık istek kısıtlamalarına (Rate Limiting) takılmamak ve bant genişliğini optimize etmek amacıyla, sonsuz döngü her veri gönderiminin ardından 5 saniye bekletilerek stabil bir telemetri akışı sağlanmıştır.

```
client.tls_set(ca_certs=root_ca_path,
               certfile=cert_path,
               keyfile=key_path,
               tls_version=ssl.PROTOCOL_TLSv1_2,
               alpn_protocols=["x-amzn-mqtt-ca"]) # <-- GİZLİ SİLAH: ALPN EKLENDİ

# AWS'ye bağlan (Port 8883 yerine standart 443 kullanıyoruz)
print("⌚ AWS'ye bağlanılıyor (Port 443 ALPN Bypass), lütfen bekleyin...")
client.connect(AWS_ENDPOINT, 443, 60) # <-- PORT 443 OLARAK DEĞİŞTİ
client.loop_start()
```

Bu kod bloğu, IoT simülatörünün AWS IoT Core bulut sunucularına bağlanırken kullandığı uçtan uca şifreleme (E2EE) ve ağ kısıtlamalarını aşma (Firewall Bypass) mekanizmalarını içermektedir.

- **Karşılıklı Kimlik Doğrulama (mTLS) ve tls_set:** paho-mqtt istemcisinin güvenlik katmanı oluşturulurken, AWS tarafından sağlanan X.509 standartlarındaki sertifikalar (Root CA, Cihaz Sertifikası ve Gizli Anahtar) sisteme entegre edilmiştir. İletişim güvenliği, endüstri standardı olan ssl.PROTOCOL_TLSv1_2 (Transport Layer Security) versiyonu ile şifrelenerek veri sızıntılarına (Sniffing/Man-in-the-Middle) karşı koruma altına alınmıştır.
- **ALPN (Application-Layer Protocol Negotiation) Entegrasyonu:** Kod bloğundaki en kritik mühendislik hamlesi, `alpn_protocols=["x-amzn-mqtt-ca"]` parametresinin kullanılmasıdır. Kurumsal ağlarda (üniversite kampüs ağları, KYK yurtları vb.) standart IoT iletişim portu olan 8883 genellikle siber güvenlik politikaları gereği kapalı tutulmaktadır. ALPN protokolü sayesinde sistem, MQTT veri paketlerini standart web trafiği gibi maskeleyebilir.
- **Port 443 Yönlendirmesi (client.connect):** Maskelenen telemetri verileri, engellenmesi mümkün olmayan standart HTTPS portu (Port 443) üzerinden AWS bitiş noktasına (Endpoint) iletilmiştir.

Bağlantı parametresindeki 60 değeri, cihazın bulut ile arasındaki bağlantının kopmaması için 60 saniyelik "Keep-Alive" (Hayatta Kalma) sinyali göndereceğini belirtir.

- **Asenkron Ağ Döngüsü (client.loop_start()):** Bağlantı sağlandıktan sonra çağrılan bu fonksiyon, ağ trafiğini (gelen ve giden mesajları) yönetmek için arka planda yeni bir iş parçası (Thread) başlatır. Bu sayede ana programın (sensör simülasyon döngüsünün) ağ gecikmelerinden dolayı kilitlenmesi (blocking) engellenmiş ve asenkron bir iletişim altyapısı kurulmuştur.